紅綠燈運行程式碼解說

```
.traffic-light {
  width: 60px;
  height: 180px;
  background-color: #333;
  border-radius: 15px;
  padding: 10px;
  margin: 10px;}
.light {
  width: 50px;
  height: 50px;
  background-color: gray;
  border-radius: 50%;
  margin: 10px auto;}
.red { background-color: red; }
.yellow { background-color: yellow; }
.green { background-color: green; }
.crossroad {
  display: flex;
  justify-content: center;
  align-items: center;
  flex-wrap: wrap;}
.direction {
  margin: 10px;
  text-align: center;}
```

紅綠燈底圖相關設定 (style)

- traffic-light
  - 紅綠燈邊框設計
- light
  - 紅綠燈未亮燈狀態
- red
  - 紅燈設定
- yellow
  - 黃燈設計
- green
  - 綠燈設計
- crossroad
  - 燈號擺放設定
- direction
  - 燈號容器設定

```
<div class="crossroad"> <!-- 十字路口設計 -->
    <div class="direction" id="north">
      <div class="traffic-light">
        <div class="light red" id="north-red"></div>
        <div class="light" id="north-yellow"></div>
        <div class="light" id="north-green"></div>
      </div>
      <p>北</p>
    </div>

    <div class="direction" id="south">
      <div class="traffic-light">
        <div class="light red" id="south-red"></div>
        <div class="light" id="south-yellow"></div>
        <div class="light" id="south-green"></div>
      </div>
      <p>南</p>
    </div>

    <div class="direction" id="east">
      <div class="traffic-light">
        <div class="light red" id="east-red"></div>
        <div class="light" id="east-yellow"></div>
        <div class="light" id="east-green"></div>
      </div>
      <p>東</p>
    </div>

    <div class="direction" id="west">
      <div class="traffic-light">
        <div class="light red" id="west-red"></div>
        <div class="light" id="west-yellow"></div>
        <div class="light" id="west-green"></div>
      </div>
```

紅綠燈展示設定(body)

- 分成東、西、南、北共四個方向的燈號，並依照crossroad的設定擺放
- 將每個方向燈號各自依照direction的設定放入容器內，並設定其ID
- 將每個方向的三個燈號置於traffic-light內，並再設定紅、黃、綠共三個燈號與其ID

```
      <p>西</p>
    </div>
  </div>
```

```
$(document).ready(function() {
  let nsGreenTime = 10000; // 南北方向綠燈 10 秒
  let ewGreenTime = 10000; // 東西方向綠燈 10 秒
  let yellowTime = 3000;   // 黃燈 3 秒

  function setLightState(direction, redOn, yellowOn, greenOn)
{
    $(`#${direction}-red`).removeClass('red'); // 重置當前方向的
燈號顏色
    $(`#${direction}-yellow`).removeClass('yellow');
    $(`#${direction}-green`).removeClass('green');

    // 根據狀態參數添加顏色
    if (redOn) {
      $(`#${direction}-red`).addClass('red');
    }
    if (yellowOn) {
      $(`#${direction}-yellow`).addClass('yellow');
    }
    if (greenOn) {
      $(`#${direction}-green`).addClass('green');
    }
  }

  function startTrafficCycle() {
    console.log('開始南北方向綠燈');

    // 南北方向綠燈階段，東西方向紅燈
    setLightState('north', false, false, true); // 北綠燈
    setLightState('south', false, false, true); // 南綠燈
    setLightState('east', true, false, false);  // 東紅燈
    setLightState('west', true, false, false);  // 西紅燈

    setTimeout(function() {
      console.log('南北方向黃燈階段');

      // 南北方向黃燈階段
      setLightState('north', false, true, false);
      setLightState('south', false, true, false);

      setTimeout(function() {
        console.log('東西方向綠燈階段');

        // 南北方向紅燈，東西方向綠燈
        setLightState('north', true, false, false);
        setLightState('south', true, false, false);
        setLightState('east', false, false, true);  // 東綠燈
        setLightState('west', false, false, true);  // 西綠燈

        setTimeout(function() {
          console.log('東西方向黃燈階段');

          setLightState('east', false, true, false); // 東西方向
黃
          setLightState('west', false, true, false);

          setTimeout(startTrafficCycle, yellowTime); // 循環重新
開始
        }, ewGreenTime);
```

燈號運作程式碼(script)

- 事先設定運作秒數
  - 綠燈設定十秒
  - 黃燈設定三秒
- setLightState()
  - 共有diretion、redOn、yellowOn、greenOn四項參數
  - diretion對應到四個方向(東、西、南、北)
  - redOn、yellowOn、greenOn都是布林值，分別控制三種燈號的開關
  - removeclass()負責控制將燈號當前狀況移除
  - addclass()依照redOn、yellowOn、greenOn的狀態控制燈號狀態
- startLightState()負責控制燈號循環
  - 開始時，南北向顯示綠燈，東西向顯示紅燈
  - 南北向綠燈(10秒)過後，南北向改為顯示黃燈
  - 南北向黃燈(3秒)過後，南北向改為紅燈，東西向改為綠燈
  - 後續燈號運行則以此類推
  - 運行前將所有燈號轉為紅燈，然後再開始進行燈號運型與循環

```
      }, yellowTime);
    }, nsGreenTime);
  }

  setLightState('north', true, false, false);
  setLightState('south', true, false, false);
  setLightState('east', true, false, false);
  setLightState('west', true, false, false);

  startTrafficCycle(); // 紅綠燈運轉開始
});
```

照片蒐集與統計程式碼講解

```
body { font-family: Arial, sans-serif; margin: 20px;
}
#data-table { margin-top: 20px; }
table { width: 100%; border-collapse: collapse; }
th, td { padding: 10px; border: 1px solid #ccc; }
th { background-color: #f4f4f4; }
.delete-btn { color: red; cursor: pointer; }
#chartContainer { margin-top: 20px; }
```

頁面基本設定(style)

- body的字體設定成Arial，並使用20px的邊距
- table的設定成表格
- delete-btn設定資料刪除按鈕的樣式
- chartContainer設定圖表容器

```
<h1>資料收集工具</h1>

<form id="dataForm">   <!-- 數據輸入表單 -->
  <label for="dataText">名稱輸入：</label>
  <input type="text" id="dataText" name="dataText"
required><br><br>

  <label for="dataDate">選擇日期：</label>
  <input type="date" id="dataDate" name="dataDate"
required><br><br>

  <label for="dataImage">上傳檔案：</label>
  <input type="file" id="dataImage" name="dataImage"
accept="image/*"><br><br>

  <button type="submit">新增資料</button>
</form>
```

資料輸入區設定(body)

- form設定圖表輸入處
- 共可以輸入資料的名稱、上傳日期
- 上傳檔案處可以選擇檔案
- 最後可以透過新增資料的按鈕上傳

```
<div id="data-table"> <!-- 顯示資料的表格 -->
  <h2>已收集數據</h2>
  <table id="tableData">
    <thead>
      <tr>
        <th>文字</th>
        <th>日期</th>
        <th>圖片</th>
        <th>操作</th>
```

圖表設計(body)

- 設定table可以顯示照片的名稱、上傳日期、照片縮圖與刪除按鈕

```
      </tr>
    </thead>
    <tbody>
    </tbody>
  </table>
</div>
```

```
$(document).ready(function() {
  let dataEntries = [];
  let ctx = $('#dataChart'); //圖表初始化
  let dataChart = new Chart(ctx, {
    type: 'bar',
    data: {
      labels: [],
      datasets: [{
        label: '每個日期的輸入次數',
        data: [],
        backgroundColor: 'rgba(75, 192, 192, 0.2)',
        borderColor: 'rgba(75, 192, 192, 1)',
        borderWidth: 1
      }]
    },
    options: {
      scales: {
        y: { beginAtZero: true }
      }
    }
  });

  function updateChart() { // 動態更新圖表的function
    let dateCounts = {};
    dataEntries.forEach(entry => {
      dateCounts[entry.date] =
(dateCounts[entry.date] || 0) + 1;
      });

    let dates = Object.keys(dateCounts);
    let counts = Object.values(dateCounts);

    dataChart.data.labels = dates;
    dataChart.data.datasets[0].data = counts;
    dataChart.update();}

  function addData(dataText, dataDate, dataImage) {
// 新增數據的function
    let newEntry = { text: dataText, date: dataDate,
image: dataImage };
    dataEntries.push(newEntry);

    // 將輸入資料新增到表格中
    let row = `<tr>
      <td>${dataText}</td>
      <td>${dataDate}</td>
      <td>${dataImage ? `<img
src="${URL.createObjectURL(dataImage)}" width="50"
/>` : '無圖片'}</td>
      <td><span class="delete-btn">刪除</span></td>
      </tr>`;
    $('#tableData tbody').append(row);

      updateChart(); // 更新圖表
    }
```

照片上傳程式碼(script)

- 初始設定一個圖表，樣式設定成柱狀圖
- labels設定成x軸的標籤，會依照使用者所選擇的上傳日期做變化
- datasets設定圖表相關樣式設定
- scales設定y軸從0開始計算
- updatdChart()負責圖表的更新
  - datecount負責日期的檔案數量計算
  - dates與counts設定成日期與其對應的資料數量
  - labels與data設為dates與counts
  - dataChart.update()負責圖表重新渲染
  - addData()共有三個參數，datatext、datadate、dataimage，包含資料的名稱、日期與相片，並將資料添加進去dataEntries內並更新表格內容與圖表
  - 最後設定資料刪除功能，資料刪除後，圖表與表格樣式更新

```
  $('#dataForm').on('submit', function(e) { // 表單提交處理
    e.preventDefault();

    let dataText = $('#dataText').val();
    let dataDate = $('#dataDate').val();
    let dataImage = $('#dataImage')[0].files[0];

    addData(dataText, dataDate, dataImage);

    $('#dataText').val(''); // 清空表單
    $('#dataDate').val('');
    $('#dataImage').val('');});

  $('#tableData').on('click', '.delete-btn', function() { // 刪除數據的function
    let rowIndex = $(this).closest('tr').index();
    dataEntries.splice(rowIndex, 1); // 刪除數據
    $(this).closest('tr').remove();  // 刪除表格行

    updateChart(); // 刪除後更新圖表
  });
});
```